CS301: Computability and Complexity Theory (CC)

Lecture 13: Space Complexity

Dumitru Bogdan

Faculty of Computer Science
University of Bucharest

January 17, 2025

Table of contents

1. Previously on CS301
2. Context setup
3. Space Complexity
4. Savitch's Theorem
5. Class PSPACE
6. PSPACE-completeness

Section 1

Previously on CS301

NP-Completeness

- One important advance on the *P versus NP question* came in the early 1970s with the work of Stephen Cook and Leonid Levin
- They discovered certain problems in NP whose individual complexity is related to that of the entire class
- If a polynomial time algorithm exists for any of these problems, all problems in NP would be polynomial time solvable
- These problems are called **NP-complete**. The phenomenon of NP-completeness is important for both theoretical and practical reasons

Reducibility

Definition

A function $f : \Sigma^* \to \Sigma^*$ is a **polynomial time computable function** if some polynomial time TM M exists that halts with just $f(w)$ on its tape, when started on any input w

Definition

Language A is **polynomial time mapping reducible**, or simply **polynomial time reducible**, to language B, written $A \leq_P B$, if a polynomial time computable function $f : \Sigma^* \to \Sigma^*$ exists, where for every w

$$w \in A \iff f(w) \in B$$

The function f is called the **polynomial time reduction** of A to B

NP-Completeness

Definition

A language B is **NP-complete** if it satisfies two conditions:

- B is in NP
- every A in NP is polynomial time reducible to B

Theorem

If B is NP-complete and $B \leq_P C$ for C in NP, then C is NP-complete

SAT is NP-complete

The Cook-Levin theorem

Once we have one NP-complete problem, we may obtain others by polynomial time reduction from it. However, establishing the first NP-complete problem is more difficult. Now we do so by proving that SAT is NP-complete.

Theorem

SAT is NP-complete

SAT is NP-complete

Proof idea: Showing that *SAT* is in *NP* is easy. The hard part of the proof is showing that any language in *NP* is polynomial time reducible to *SAT*.

To do so, we construct a polynomial time reduction for each language A in *NP* to *SAT*. The reduction for A takes a string w and produces a boolean formula ϕ that simulates the NP machine for A on input w. If the machine accepts, ϕ has a satisfying assignment that corresponds to the accepting computation. If the machine doesn't accept, no assignment satisfies ϕ. Therefore, w is in A if and only if ϕ is satisfiable.

Actually constructing the reduction to work in this way is a conceptually simple task and the formula ϕ is the AND of four parts:

$$\phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

Section 2

Context setup

Context setup

Corresponding to 8.1

Context setup

- Now we consider the complexity of computational problems in terms of the amount of space, or memory, that they require
- Time and space are two of the most important considerations when we seek practical solutions to many computational problems
- Space complexity shares many of the features of time complexity and serves as a further way of classifying problems according to their computational difficulty

Section 3

Space Complexity

Space Complexity

As we did with time complexity, we need to select a model for measuring the space used by an algorithm. We continue with the TM model for the same reason that we used it to measure time. TMs are mathematically simple and close enough to real computers to give meaningful results.

Definition

Let M be a deterministic TM that halts on all inputs. The space complexity of M is the function $f : N \to N$, where $f(n)$ is the maximum number of tape cells that M scans on any input of length n. If the space complexity of M is $f(n)$, we also say that M runs in space $f(n)$. If M is a nondeterministic TM wherein all branches halt on all inputs, we define its space complexity $f(n)$ to be the maximum number of tape cells that M scans on any branch of its computation for any input of length n.

Space Complexity

We typically estimate the space complexity of TMs by using asymptotic notation.

Definition

Let $f : N \to R^+$ be a function. The space complexity classes, SPACE(f(n)) and NSPACE(f(n)), are defined as follows:

$SPACE(f(n)) = \{L \mid L \text{ is a language decided by an } O(f(n)) \text{ space deterministic TM}\}$

$NSPACE(f(n)) = \{L \mid L \text{ is a language decided by an } O(f(n)) \text{ space nondeterministic TM}\}$

Example I

In previously chapters, we introduced the NP-complete problem SAT. We show that SAT can be solved with a linear space algorithm. We believe that SAT cannot be solved with a polynomial time algorithm, much less with a linear time algorithm, because SAT is NP-complete.

Space appears to be more powerful than time because space can be reused, whereas time cannot.

Example I

M_1 = On input $< \phi >$, where ϕ is a Boolean formula:

1. For each truth assignment to the variables $x_1, x_2, \ldots, x_m$ of ϕ:
2. Evaluate ϕ on that truth assignment.
3. If ϕ ever evaluated to 1, accept ; if not, reject.

Machine M_1 clearly runs in linear space because each iteration of the loop can reuse the same portion of the tape. The machine needs to store only the current truth assignment, and that can be done with $O(m)$ space. The number of variables m is at most n, the length of the input, so this machine runs in space $O(n)$.

Example II

Next, yet us consider the problem of testing whether a nondeterministic finite automaton accepts all strings. Let:

$$ALL_{NFA} = \{< A > \mid A \text{ is a NFA and } L(A) = \Sigma^*\}$$

We give a nondeterministic linear space algorithm that decides the complement of this language $\overline{ALL_{NFA}}$. The idea behind this algorithm is to use nondeterminism to guess a string that is rejected by the NFA, and to use linear space to keep track of which states the NFA could be in at a particular time. Note that this language is not known to be in NP or in coNP.

Example II

N = On input $< M >$, where M is a NFA:

1. Place a marker on the start state of the NFA
2. Repeat 2^q times, where q is the number of states of M:
3. Nondeterministically select an input symbol and change the positions of the markers on M's states to simulate reading that symbol
4. Accept if stages 2 and 3 reveal some string that M rejects; that is, if at some point none of the markers lie on accept states of M. Otherwise, reject.

Example III

If M rejects any strings, it must reject one of length at most 2^q because in any longer string that is rejected, the locations of the markers described in the preceding algorithm would repeat. The section of the string between the repetitions can be removed to obtain a shorter rejected string. Hence N decides $\overline{ALL_{NFA}}$. (Note that N accepts improperly formed inputs, too.)

The only space needed by this algorithm is for storing the location of the markers and the repeat loop counter, and that can be done with linear space. Hence the algorithm runs in nondeterministic space $O(n)$. Next, we prove a theorem that provides information about the deterministic space complexity of ALL_{NFA}.

Section 4

Savitch's Theorem

Savitch's theorem

- Savitch's theorem is one of the earliest results concerning space complexity
- It shows that deterministic machines can simulate nondeterministic machines by using a surprisingly small amount of space
- For time complexity, such a simulation seems to require an exponential increase in time
- For space complexity, Savitch's theorem shows that any nondeterministic TM that uses $f(n)$ space can be converted to a deterministic TM that uses only $f^2(n)$ space

Savitch's theorem

Theorem

For any function $f : N \to \mathcal{R}^+$, *where* $f(n) \geq n$, $NSPACE(f(n)) \subseteq SPACE(f^2(n))$

Proof idea: We need to simulate an $f(n)$ space NTM deterministically. A naive approach is to proceed by trying all the branches of the NTM's computation, one by one. The simulation needs to keep track of which branch it is currently trying so that it is able to go on to the next one. But a branch that uses $f(n)$ space may run for $2^{O(f(n))}$ steps and each step may be a nondeterministic choice. Exploring the branches sequentially would require recording all the choices used on a particular branch in order to be able to find the next branch. Therefore, this approach may use $2^{O(f(n))}$ space, exceeding our goal of $O(f^2(n))$ space.

Savitch's theorem

Proof idea: Instead, we take a different approach by considering the following more general problem. We are given two configurations of the NTM, c_1 and c_2, together with a number t, and we test whether the NTM can get from c_1 to c_2 within t steps using only $f(n)$ space. We call this problem the **yieldability problem**. By solving the yieldability problem, where c_1 is the start configuration, c_2 is the accept configuration, and t is the maximum number of steps that the nondeterministic machine can use, we can determine whether the machine accepts its input.

Savitch's theorem

We give a deterministic, recursive algorithm that solves the yieldability problem. It operates by searching for an intermediate configuration c_m, and recursively testing whether:

1. c_1 can get to cm within $t/2$ steps
2. whether c_m can get to c_2 within $t/2$ steps

Reusing the space for each of the two recursive tests allows a significant savings of space.

This algorithm needs space for storing the recursion stack. Each level of the recursion uses $O(f(n))$ space to store a configuration. The depth of the recursion is $log\, t$, where t is the maximum time that the nondeterministic machine may use on any branch. We have $t = 2^{O(f(n))}$, so $log\, t = O(f(n))$. Hence the deterministic simulation uses $O(f^2(n))$ space.

Savitch's theorem

Proof.

Let N be an NTM deciding a language A in space $f(n)$. We construct a deterministic TM M deciding A. Machine M uses the procedure *CANYIELD*, which tests whether one of N's configurations can yield another within a specified number of steps. This procedure solves the yieldability problem described in the proof idea. Let w be a string considered as input to N.

For configurations c_1 and c_2 of N, and integer t, $CANYIELD(c_1, c_2, t)$ outputs accept if N can go from configuration c_1 to configuration c_2 in t or fewer steps along some nondeterministic path. If not, *CANYIELD* outputs reject . For convenience, we assume that t is a power of two. □

Savitch's theorem

Proof.

N = On input c_1, c_2 and t:

1. Place a marker on the start state of the NFA
2. If $t = 1$, then test directly whether $c_1 = c_2$ or whether c_1 yields c_2 in one step according to the rules of N. Accept if either test succeeds; reject if both fail.
3. If $t > 1$, then for each configuration c_m of N using space $f(n)$:
4. Run $CANYIELD(c_1, c_m, t/2)$
5. Run $CANYIELD(c_m, c_2, t/2)$
6. If steps 3 and 4 both accept, then accept
7. If haven't yet accepted, reject

□

Savitch's theorem

Proof.

Next we define M to simulate N as follows. We first modify N so that when it accepts, it clears its tape and moves the head to the leftmost cell—thereby entering a configuration called c_{accept}. We let c_{start} be the start configuration of N on w. We select a constant d so that N has no more than $2^{df(n)}$ configurations using $f(n)$ tape, where n is the length of w. Then we know that $2^{df(n)}$ provides an upper bound on the running time of any branch of N on w.

$M =$ On input w:

1. Output the result of $CANYIELD(c_{start}, c_{accept}, 2^{df(n)})$

□

Savitch's theorem

Proof.

Algorithm *CANYIELD* obviously solves the yieldability problem, and hence M correctly simulates N. We need to analyze it to verify that M works within $O(f^2(n))$ space. Whenever *CANYIELD* invokes itself recursively, it stores the current stage number and the values of c_1, c_2, and t on a stack so that these values may be restored upon return from the recursive invocation. Each level of the recursion thus uses $O(f(n))$ additional space. Furthermore, each level of the recursion divides the size of t in half. Initially t starts out equal to $2^{df(n)}$, so the depth of the recursion is $O(log 2^{df(n)})$ or $O(f(n))$. Therefore, the total space used is $O(f^2(n))$, as claimed. □

Section 5

Class PSPACE

Class PSPACE

By analogy with the class P, we define the class $PSPACE$ for space complexity:

Definition

PSPACE is the class of languages that are decidable in polynomial space on a deterministic TM. In other words:

$$PSPACE = \bigcup_k SPACE(n^k)$$

We define $NPSPACE$, the nondeterministic counterpart to $PSPACE$, in terms of the $NSPACE$ classes. However, $PSPACE = NPSPACE$ by virtue of Savitch's theorem because the square of any polynomial is still a polynomial.

P,NP,PSPACE

Let's examine the relationship of *PSPACE* with *P* and *NP*. We observe that $P \subseteq PSPACE$ because a machine that runs quickly cannot use a great deal of space. More precisely, for $t(n) \geq n$, any machine that operates in time $t(n)$ can use at most $t(n)$ space because a machine can explore at most one new cell at each step of its computation. Similarly, $NP \subseteq NPSPACE$, and so $NP \subseteq PSPACE$.

Conversely, we can bound the time complexity of a TM in terms of its space complexity. For $f(n) \geq n$, a TM that uses $f(n)$ space can have at most $f(n)2^{O(f(n))}$ different configurations. A TM computation that halts may not repeat a configuration. Therefore, a TM that uses space $f(n)$ must run in time $f(n)2^{O(f(n))}$ so we have:

$$P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXPTIME$$

P,NP,PSPACE

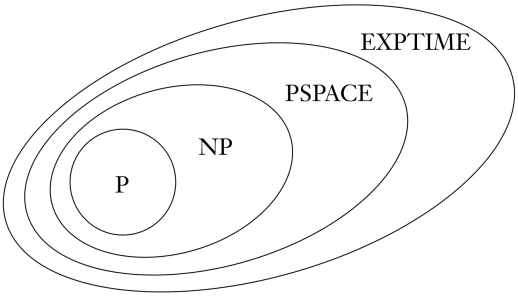


Figure: Conjectured relationships

We don't know whether any of these containments is actually an equality. Someone may yet discover a simulation like the one in Savitch's theorem that merges some of these classes into the same class. However, we prove that $P \neq EXPTIME$. Therefore, at least one of the preceding containments is proper, but we are unable to say which!

Section 6

PSPACE-completeness

PSPACE-completeness

Previously we have introduced the category of $NP - complete$ languages as representing the most difficult languages in NP. Demonstrating that a language is $NP - complete$ provides strong evidence that the language is not in P. If it were, P and NP would be equal. In this section, we introduce the analogous notion $PSPACE - completeness$ for the class $PSPACE$

Definition

A language B is PSPACE-complete if it satisfies two conditions:

1. B is in PSPACE
2. every A in PSPACE is polynomial time reducible to B

PSPACE-completeness

In defining PSPACE-completeness, we use polynomial time reducibility. Why?

- Complete problems are important because they are examples of the most difficult problems in a complexity class
- A complete problem is most difficult because any other problem in the class is easily reduced into it
- The reduction must be easy, relative to the complexity of typical problems in the class, for this reasoning to apply
- If the reduction itself were difficult to compute, an easy solution to the complete problem wouldn't necessarily yield an easy solution to the problems reducing to it

The rule is

Whenever we define complete problems for a complexity class, the reduction model must be more limited than the model used for defining the class itself

PSPACE-completeness

We now introduce a more general type of Boolean formula because a first example of a PSPACE-complete problem involves a generalization of the satisfiability problem.

- The quantifiers $\forall$ (for all) and $\exists$ (there exists) make frequent appearances in mathematical statements. Writing the statement $\forall x \phi$ means that for every value for the variable x, the statement ϕ is true
- Similarly, writing the statement $\exists x \phi$ means that for some value of the variable x, the statement ϕ is true
- Sometimes, $\forall$ is referred to as the universal quantifier and $\exists$ as the existential quantifier
- We say that the variable x immediately following the quantifier is bound to the quantifier

Boolean formulas with quantifiers are called **quantified Boolean formulas**

$$\phi = \forall x \exists y [(x \vee y) \wedge (\overline{x} \vee \overline{y})]$$

TQBF

- When each variable of a formula appears within the scope of some quantifier, the formula is said to be **fully quantified**
- A fully quantified Boolean formula is sometimes called a sentence and is always either true or false

The TQBF problem is to determine whether a fully quantified Boolean formula is true or false. We define the language

$$TQBF = \{< \phi > \mid \phi \textit{ is a true fully quantified boolean formula}\}$$